

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 6 — Projects e Context Engineering

Il laboratorio: un CLAUDE.md per il gestionale o per la rete

Il laboratorio: un CLAUDE.md per il gestionale o per la rete

Due tracce, un metodo: le stesse richieste senza il file e con due versioni

- Il **laboratorio** del Modulo 6. Un `CLAUDE.md` è il file di istruzioni che Claude Code carica all'inizio di ogni sessione: qui ne scrivete uno per un progetto sintetico di CiaspoLink, l'operatore inventato del corso, e lo provate **come il codice**. In tutto 59 minuti
- **Due tracce, una a testa**, con gli stessi passi e lo stesso prompt: M6-E02, il **gestionale** PHP, per chi sviluppa (Docker, oppure PHP con Composer: `tools/php.sh` sceglie da solo); M6-E03, la **rete**, per chi gestisce sistemi e reti (soltanto Python 3)
- I README, con i passi e i prompt: `m6_projects_context/s5_projects_context/m6_e2_laboratorio/`, poi `M6-E02_claude_md_gestionale/` · `M6-E03_claude_md_rete/`
- Il metodo, dall'Academy di Anthropic, «The CLAUDE.md file»: «We recommend starting a project without a CLAUDE.md file so you can see where you constantly have to course-correct the model.»
- Che cosa scriverci, dalle «Best practices for Claude Code»: «Only write down what Claude can't find by reading the repo, or what it would otherwise spend time rediscovering every session.» Le **due domande**: «Could it find this by reading the code?» e «Would it actually go and look?»; «If both are yes, leave it out.»
- E ciò che avete dovuto correggere due volte, dal playbook dell'Academy: «When Claude makes a mistake twice, the correction goes into CLAUDE.md.»

Le tre richieste

Le stesse in `base`, senza `CLAUDE.md`, e nelle due versioni, v1 e v2

	il gestionale, M6-E02	la rete, M6-E03
i controlli del progetto	t1 «Esegui la suite di test del gestionale e dimmi quanti test passano e quanti falliscono.»	r1 «Controlla che i file del progetto rete siano a posto e dimmi l'esito.»
un refuso, e il commit	t2 «Nel messaggio d'errore di bin/fatturazione-mensile.php c'è un refuso: correggilo e fai il commit.»	r2 «Nella checklist di audit c'è un refuso nello standard di AUD-02: correggilo e fai il commit.»
una domanda che col `CLAUDE.md` non c'entra	t3 «Che cosa fa src/Copertura/VerificaCopertura.php? Rispondi in cinque righe.»	r3 «Che cosa controlla la categoria AUD-07 della checklist di audit? Rispondi in cinque righe.»

- Che cosa legge il **controllo**, dalla trascrizione e dai file: nella prima, quali comandi Claude ha lanciato e se le cifre della risposta sono le loro; nella seconda, il refuso, il commit con il suo soggetto e la riga `Rif:`, i file toccati; nella terza, la risposta, e i token della prima chiamata al modello
- Perché queste tre. Nel `CLAUDE.md` vanno «the exact test, typecheck, and lint commands» e «Decisions that aren't in the repo»; e ogni sua riga «takes a little of Claude's attention on every task, whether it's relevant or not.»
- Scritte come le scriverebbe chiunque, stanno in `prove/richieste.tsv` dell'elemento, con gli strumenti di ogni richiesta: uguali per le tre versioni

Prima, senza CLAUDE.md

La prova base, e intanto gli appunti del team: sei minuti

```
# dalla cartella esercitazioni; <ID>: M6-E02, il gestionale,
# oppure M6-E03, la rete
tools/prepara_esercizio.sh <ID>
tools/prova_contesto.sh <ID> base
```

- Le tre richieste, ognuna in una **sessione nuova** di `claude -p`, in una copia pulita dello spazio di lavoro, senza `CLAUDE.md` né regole; al massimo tre insieme. Fuori da `base`, `claude -p` carica i file di istruzioni come una sessione interattiva: «claude -p discovers and loads them as an interactive session would»
- L'**auto memory**, la memoria automatica del Modulo 4, è spenta in ogni prova, con `CLAUDE_CODE_DISABLE_AUTO_MEMORY`: «Set to 1 to disable auto memory.» Così le versioni differiscono soltanto nei file di istruzioni
- Mentre girano: `~/corso-claude-code/<ID>/appunti-del-team.md`. Per ogni appunto, le **due domande**: Claude lo troverebbe leggendo il repository? Andrebbe a cercarlo?
- Alla fine lo strumento stampa ciò che il controllo ha letto: guardate che cosa Claude ha trovato **da solo**. I comandi che ha lanciato stanno nella trascrizione, `prove-contesto/base/t1/stream.jsonl`, `r1` nella rete

La prima versione

`/init`, poi il prompt che asciuga la bozza: venti minuti

```
# in un altro terminale, su Sonnet con effort medium; o M6-E03
cd ~/corso-claude-code/M6-E02
claude --model sonnet --effort medium
# nella sessione: /init, poi il prompt 1 del README
# quando il file c'è, nel primo terminale: si carica?
cd ~/corso-claude-code/M6-E02
claude -p "/context all"
cd -
```

- La prima volta vi chiede se vi **fidate della cartella**: «In an interactive session, Claude Code shows the workspace trust dialog when you start it in a folder you haven’t trusted.» Sì: il cursore parte da «No, exit», scendete con la freccia su «Yes, I trust this folder» e date Invio
- `/init` scrive la bozza: «Initialize project with a CLAUDE.md guide.» Poi incollate il **prompt 1** del README, lo stesso nelle due tracce: nelle pagine che seguono. Leggete la modifica prima di approvarla, e **lasciate aperta** la sessione: serve alla seconda versione
- Il file si carica nelle **sessioni nuove**, non in quella dove è nato: «Your project-root and user-level CLAUDE.md files are read once at session start and held in memory.» Con `/context all` lo vedete sotto «Memory files», con i suoi token: «To confirm the file loaded, run /context all in a session and check the list under Memory files.»

La prima versione

`/init`, poi il prompt che asciuga la bozza: venti minuti

il prompt 1 del README, 1 di 3

```
<role>
Sei un collega del team che cura il progetto in progetti/ e
scrivi il CLAUDE.md che i colleghi caricano a ogni sessione di
Claude Code.
</role>

<context>
Il CLAUDE.md alla radice di questa cartella l'ha appena scritto
/init: è la bozza da asciugare. In appunti-del-team.md ci sono gli
appunti del team, che il repository non contiene: decisioni prese
a voce, abitudini, procedure, preferenze. Il CLAUDE.md entra nel
contesto all'inizio di ogni sessione, e ogni sua riga pesa su ogni
richiesta: tieni solo ciò che cambia quello che Claude fa.
</context>
```

Lo stesso nelle due tracce: il metodo non cambia con il progetto. Si copia dal **README**, mai dalla slide

La prima versione

`/init`, poi il prompt che asciuga la bozza: venti minuti

il prompt 1 del README, 2 di 3

```
<instructions>
1. Leggi il CLAUDE.md, appunti-del-team.md e i documenti che
   nominano.
2. Per ogni riga del CLAUDE.md fatti le due domande: Claude la
   troverebbe leggendo il repository? Andrebbe davvero a cercarla?
   Togli le righe con due sì.
3. Aggiungi dagli appunti ciò che il repository non dice e che
   vale per tutto il team, con i comandi esatti e la cartella da
   cui si lanciano.
4. Per ultimo conta le righe del file e rileggile una per una con
   le due domande: correggi finché il file sta in una pagina,
   sotto le 200 righe, e ogni riga ha almeno un no.
</instructions>
```

La misura del passo 4 è di Anthropic. L'Academy: «Keep it under a page, because Claude reads all of it at the start of a session». La documentazione: «target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence.»

La prima versione

`/init`, poi il prompt che asciuga la bozza: venti minuti

il prompt 1 del README, 3 di 3

```
<format>
Il CLAUDE.md con titoli Markdown ed elenchi puntati, righe
concrete e verificabili: una riga su che cos'è il progetto, poi
le sezioni Comandi, Convenzioni che il codice non dice e Insidie,
quelle che servono. Nella risposta, al massimo dieci righe: per
ogni appunto, dove va (CLAUDE.md, già nel repository, una regola
per una parte del codice, una skill, CLAUDE.local.md, una regola
deny) e perché.
</format>

<constraints>
Scrivi soltanto il CLAUDE.md alla radice. Ciò che vale per una
sola parte del codice, le procedure in più passi, le preferenze
personali e ciò che deve valere sempre restano fuori dal file:
elencali nella risposta, per il passo seguente.
</constraints>
```

Le stesse richieste, con la v1

La prova v1, e il confronto con base: dieci minuti

```
# dalla cartella esercitazioni; o M6-E03
tools/prova_contesto.sh M6-E02 v1
```

- Le stesse tre richieste, con la v1. Il **confronto**, criterio per criterio, base contro v1: **«confermata»** se senza il file no e con il file sì; **«non distinguibile»** se è uguale; **«non confermata in questa misura»** se senza sì e con no
- Si decide confrontando, perché la frase di Anthropic è un confronto: «Only write down what Claude can't find by reading the repo, or what it would otherwise spend time rediscovering every session.»
- Sotto ogni richiesta, per ogni versione: «prima richiesta … token · … turni · … s · … token»: il primo numero è il contesto della **prima chiamata al modello**, l'ultimo i token di tutta la sessione. Una riga che fa **risparmiare la ricerca** si vede lì, anche quando il criterio era già sì
- **Copiate** le righe che lo strumento stampa, da «Il confronto delle versioni» all'ultima: servono alla seconda versione. Sono **dati**, non l'esito della verifica: una sola esecuzione per versione è un campione

Le stesse richieste, con la v1

La prova v1, e il confronto con base: dieci minuti

M6-E02, a capo per la slide: «[…]» segna un taglio

```
Richiesta t2: «Nel messaggio d'errore di
  bin/fatturazione-mensile.php c'è un refuso: correggilo e fai il
  commit.»
[…]
  v1 contro base: il refuso corretto: non distinguibile; un
    commit: non distinguibile; il soggetto nella forma di
    docs/convenzioni.md, 11.1: confermata; la riga «Rif: GES-0»:
    confermata; cambiato soltanto bin/fatturazione-mensile.php:
    non distinguibile; legacy/ e database/dati/ intatti: non
    distinguibile
[…]
```

Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

- Per il soggetto nella forma del team e per la riga `Rif: GES-0`, il controllo dice **«confermata»**. Nella rete lo stesso, con `rete(<ambito>)` e `Rif: NOC-0`
- Nella prima delle tre richieste, in tutte e due le tracce, **«non distinguibile»**: il comando dei test e i due controlli della rete Claude li ha trovati anche senza il file. Con più turni e più token: nel gestionale «base: prima richiesta 6426 token · 6 turni · 15 s · 61532 token» contro «v1: … 3 turni · 19 s · 23280 token»; nella rete 4 turni e 50.441 token contro 3 e 14.303

La seconda versione

Il seguito, nella stessa sessione: una regola con `paths`, le regole `deny`; dodici minuti

- Da ciò che il controllo ha mostrato. Una riga che Claude seguiva già da solo, **senza risparmiare turni**, si toglie; una che mancava si aggiunge. Anthropic: «Each addition should solve a real problem you have encountered, not theoretical concerns about what Claude might need.»
- Ciò che vale per **una parte del codice** va in una regola di `.claude/rules/` con `paths` nel frontmatter, `legacy/` nel gestionale e `generatore/` nella rete. «If an entry … only matters for one part of the codebase, move it to a skill or a path-scoped rule»; e «A path-scoped rule loads when Claude uses the Read, Write, or Edit tool on a matching file.»
- Ciò che deve **valere sempre** non è una riga: «Claude Code (the harness) has to execute this. This can be a hook, a deny rule, or a managed setting.» Qui regole `deny` di `Edit` in `.claude/settings.json`, per `database/dati/` e `composer.lock` nel gestionale, per `configurazioni/`, `log/`, `inventario.csv` e `allarmi.csv` nella rete. «Edit rules apply to all built-in tools that edit files.»
- Con la **barra iniziale**, per la pagina dei permessi: `/path` è «Path relative to the settings source», `path` «Path relative to current directory». Per esempio `Edit(/progetti/rete/log/**)`

La seconda versione

Il seguito, nella stessa sessione: una regola con `paths`, le regole `deny`; dodici minuti

```
il prompt 2, il seguito, 1 di 3
```

```
<context>
La prima versione del CLAUDE.md è stata provata: le stesse tre
richieste, ciascuna in una sessione nuova, senza CLAUDE.md (base)
e con questa versione (v1). Che cosa il controllo ha letto nelle
esecuzioni è fra i tag verification_result, qui sotto: per ogni
richiesta i criteri in base e in v1, il confronto, turni e token.
</context>
```

- Un **seguito**, nella sessione del passo 2: niente tag `role`, il ruolo l'ha dato il primo prompt. Si copia dal README
- `.claude/` è una cartella **protetta**: quando Claude ci scrive, Claude Code vi chiede di approvare. Leggete la modifica, poi «Yes, and allow Claude to edit files in this project's .claude folder for this session». Se vi chiede di approvare un **comando**, leggete anche quello: la modifica può stare nel comando

La seconda versione

Il seguito, nella stessa sessione: una regola con `paths`, le regole `deny`; dodici minuti

il prompt 2, il seguito, 2 di 3

```
<instructions>
1. Leggi il risultato del controllo, il CLAUDE.md e
   appunti-del-team.md.
2. Per ogni criterio che con la v1 non passa, aggiungi al
   CLAUDE.md la riga che manca, o rendi più concreta quella che
   c'è.
3. Togli una riga se in base Claude ha fatto da solo ciò che
   chiede, e con la v1 non ha risparmiato turni.
4. Ciò che negli appunti vale per una sola parte del codice va in
   una regola nuova in .claude/rules/, con paths: nel frontmatter
   e i percorsi dalla radice.
5. Ciò che negli appunti deve valere sempre diventa una regola
   deny in .claude/settings.json.
6. Per ultimo rileggi i tre file: il CLAUDE.md sotto le 200 righe,
   la regola con il suo paths:, settings.json un JSON valido con
   l'hook di prima.
</instructions>
```

La seconda versione

Il seguito, nella stessa sessione: una regola con `paths`, le regole `deny`; dodici minuti

il prompt 2, il seguito, 3 di 3

```
<format>
Al massimo cinque righe: che cosa hai cambiato in ciascun file, e
quale riga del controllo o degli appunti lo chiedeva.
</format>

<constraints>
Cambia soltanto il CLAUDE.md, .claude/rules/ e
.claude/settings.json. In settings.json aggiungi le regole deny e
lascia com'è tutto il resto, l'hook compreso.
</constraints>

<verification_result>
Il confronto delle versioni …
</verification_result>
```

Fra i tag `verification_result`, al posto della riga «Il confronto delle versioni …», le righe che avete copiato dalla prova della v1

La seconda prova, e la verifica

La prova v2, la verifica, e che cosa è cambiato: sette minuti

```
# dalla cartella esercitazioni; o M6-E03
tools/prova_contesto.sh M6-E02 v2
tools/verifica.sh M6-E02
# che cosa è cambiato fra le due versioni
cd ~/corso-claude-code/M6-E02/prove-contesto
diff -rN v1/fotografia v2/fotografia
cd -
```

decide l’esito della verifica: il lavoro è fatto	**si stampa soltanto, come dato**
le tre versioni con le tre richieste, ogni sessione finita bene, un `CLAUDE.md` alla radice nella v1 e nella v2, la v2 diversa dalla v1; e la **garanzia**: in `base` nessun file di istruzioni del progetto, in `v1` e `v2` il `CLAUDE.md` caricato all’avvio, in ogni esecuzione	i criteri, richiesta per richiesta e versione per versione; i confronti, v1 contro base e v2 contro v1, con le parole «confermata», «non distinguibile», «non confermata in questa misura»; le righe e i token di ogni versione; turni, secondi e token

> La garanzia è della documentazione: «CLAUDE.md and CLAUDE.local.md files in the directory hierarchy above the working directory are loaded at launch.» La si legge nel registro dell’hook di ogni prova, non nella risposta

La seconda prova, e la verifica

La prova v2, la verifica, e che cosa è cambiato: sette minuti

- Nella **terza richiesta**, i token della prima chiamata al modello in `base`, `v1` e `v2`: la differenza è il **peso del file** su una richiesta che non lo riguarda. Anthropic: «Your CLAUDE.md file loads into the context at the start of every session, so each line in it is part of what every turn resends.»
- `VERIFICA OK` vuol dire che il lavoro è fatto, **non che il `CLAUDE.md` è buono**: quello lo dite voi, leggendo i criteri e i confronti
- Righe `MANCA`: che cosa manca, con il rimedio nel README, «Se qualcosa va storto». Righe `GARANZIA`: un file di istruzioni caricato in `base`, o il `CLAUDE.md` che non si carica; è un esito da segnalare
- Chi finisce prima: il passo 6 del README. In una sessione nuova Claude legge un file di `legacy/`, o del generatore, e l'ultima riga di `.claude/caricamenti.jsonl` dice `path_glob_match`: la regola si è caricata quando serviva

Il punto della situazione

Che cosa portate al confronto, quanto è costato, e se qualcosa è andato storto

- Al **confronto** portate le righe del controllo della vostra traccia, `base`, `v1` e `v2`, richiesta per richiesta; e una riga che avete tolto, una che avete aggiunto, e perché
- L'**impronta**: nove chiamate di prova, Sonnet con effort low; la scrittura, `/init` e due prompt, Sonnet con effort medium; nessun subagent. Il controllo e la verifica non chiamano il modello. Le prove del docente:

	le nove prove, effort low	in tutto, con la scrittura
il gestionale, M6-E02	38 turni, 94,3 s, 295.940 token	59 turni, 165,7 s, 768.537 token
la rete, M6-E03	34 turni, 72,8 s, 228.504 token	46 turni, 132,4 s, 626.132 token

Le cifre del README di ogni elemento, «Impronta d'uso»: i token di tutti i modelli, la cache compresa
Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet

- Il limite è lo stesso di claude.ai: «These limits are shared across Claude and Claude Code, meaning all activity in both tools counts against the same limits.» A che punto siete lo dice `/usage`: «Show session cost, plan usage limits, and activity stats.»
- Se qualcosa è andato storto: `/context all` in una sessione nuova; il registro dell'hook di ogni prova, `prove-contesto/<versione>/<richiesta>/caricamenti.jsonl`; la fotografia di ogni versione, `prove-contesto/<versione>/fotografia/`; il README, «Se qualcosa va storto»